Game Theory for ICPC

Oct 30, 2026 · @Nihal

1. Core idea: winning and losing positions

Most ICPC game problems are two-player games with perfect information and optimal play. The question is always the same: does the first player win from this position?

Every position is one of two kinds:

- **P-position (Previous player wins)**: the player to move LOSES with optimal play.
- **N-position (Next player wins)**: the player to move WINS with optimal play.

The two rules that decide everything:

1. A position is **losing** if EVERY move leads to a winning position. A position with no moves is losing under the normal rule ("the player who cannot move loses").
2. A position is **winning** if AT LEAST ONE move leads to a losing position.

So you can always compute it with DP or memoized recursion when the state space is small:

```
// win[s] = true if the player to move from state s wins
int memo[MAXS]; // -1 = unknown
bool win(int s) {
    if (memo[s] != -1) return memo[s];
    for (int t : moves(s))          // try every move
        if (!win(t)) return memo[s] = 1;   // found a move to a losing state
    return memo[s] = 0;             // every move gives the opponent a win
}
```

How to recognize the game type

Type	What it looks like	Tool
Impartial	Both players have the same moves; only the position matters	Win/lose DP, Nim, Sprague-Grundy
Sum of games	Several independent piles/boards; each turn you move in exactly one	Sprague-Grundy: XOR of grundy values
Score game	Players collect points; maximize your total or the difference	Minimax / interval DP

Type	What it looks like	Tool
Game on a graph with cycles	A token moves on a graph that can loop; answer can be Draw	Retrograde BFS
Partizan	Players have different moves (e.g. only White moves white pieces)	Usually plain DP on the state

Always check the ending rule first. "Last player to move wins" is the normal rule. "Last player to move loses" is called misère and often changes the answer.

2. Classic games with a formula

Memorize these. Many contest problems are one of them in disguise.

Game	Rules	First player LOSES when
Subtraction game (Bash)	One pile of n, take 1..k stones	n % (k+1) == 0
Nim	Many piles, take any number from one pile	a1 ^ a2 ^ ... ^ an == 0
Misère Nim	Nim, but the player who takes the last stone LOSES	All piles ⩽ 1: count of 1-piles is odd. Otherwise: XOR == 0
Staircase Nim	Coins on steps 0..n; move any coins from step i to step i-1	XOR of piles on ODD steps == 0
Moore's Nim-k	Many piles; take from up to k piles in one move	For every bit, (count of piles with that bit set) % (k+1) == 0
Nim with limit k	Many piles; take 1..k from one pile	XOR of (ai % (k+1)) == 0
Wythoff's game	Two piles; take any amount from one pile, or the same amount from both	(a, b) with a ⩽ b and a == floor((b - a) × φ), φ = (1+√5)/2
Euclid's game	(a, b), a ⩾ b; subtract a positive multiple of b from a; who makes a zero wins	See code below
Poker Nim	Nim, but you may also ADD stones to a pile (limited times)	Same as Nim: XOR == 0 (adding can always be undone)

Game	Rules	First player LOSES when
Northcott's game	Each row has a white and a black token; move your token toward the other	XOR of the gaps == 0 (gaps act as Nim piles)

Why Nim works: from XOR = 0 every move makes XOR ≠ 0. From XOR ≠ 0 there is always a move back to XOR = 0. The winning move: let x = XOR of all piles; find a pile with ai ^ x < ai and reduce it to ai ^ x.

```
// Nim: returns true if the first player wins; also prints a winning move
bool nimWins(vector<ll>& a) {
    ll x = 0;
    for (ll v : a) x ^= v;
    if (x == 0) return false;               // P-position: first player loses
    for (int i = 0; i < (int)a.size(); i++)
        if ((a[i] ^ x) < a[i]) {            // this pile can fix the XOR
            // winning move: reduce pile i from a[i] to a[i]^x
            break;
        }
    return true;
}

// Misère Nim
bool misereNimWins(const vector<ll>& a) {
    ll x = 0; bool big = false; int ones = 0;
    for (ll v : a) { x ^= v; if (v > 1) big = true; if (v == 1) ones++; }
    if (!big) return ones % 2 == 0;          // only 1-piles: you want an
even count
    return x != 0;                          // otherwise same as normal Nim
}

// Wythoff: true if the first player wins
bool wythoffWins(ll a, ll b) {
    if (a > b) swap(a, b);
    long double phi = (1 + sqrtl(5.0L)) / 2;
    return a != (ll)floorl((b - a) * phi);
}

// Euclid's game: (a, b) with a >= b > 0; true if the player to move wins
bool euclidWins(ll a, ll b) {
    if (a < b) swap(a, b);
```

```
    if (b == 0) return false;               // game already over
    if (a % b == 0 || a >= 2 * b) return true; // you control the outcome
    return !euclidWins(b, a - b);           // only one move possible
}
```

Two tricks that need no formula

- **Symmetry (mirror) strategy**: the second player copies the first player's move on the opposite side. Example: n coins in a circle, take 1 or 2 adjacent coins. For n ⩾ 3 the second player wins by mirroring. Often the first player wins by making the first move in the CENTER and then mirroring.
- **Strategy stealing**: if an extra "pass" move could never hurt you, the first player cannot lose. Example: Chomp on any board bigger than 1×1 is a first player win. Note it proves who wins but does not give the move.

3. Sprague-Grundy theorem (the most important tool)

Every impartial game position has a **grundy number** g. The position is losing exactly when g = 0. If a game is a SUM of independent games (each turn you move in exactly one of them), the grundy number of the sum is the XOR of the parts.

- **mex** (minimum excludant) of a set = the smallest non-negative integer NOT in the set. mex{0, 1, 3} = 2, mex{1, 2} = 0, mex{} = 0.
- g(position) = mex of g(every position you can move to).
- A terminal position (no moves) has g = 0.
- A Nim pile of size n has g = n. That is why Nim is "XOR the piles".
- A move that SPLITS one game into two independent games leads to a position with g = g(left) ^ g(right).

Recipe for almost any problem: compute g for ONE pile/component, XOR them over all components, answer "First" if the XOR is not 0.

```
// Grundy numbers for one pile, for a game where from n you can remove any
amount in set S
// (change the inner loop to describe YOUR game's moves)
vector<int> grundy(int maxN, const vector<int>& S) {
    vector<int> g(maxN + 1, 0);
    for (int n = 1; n <= maxN; n++) {
        vector<bool> seen(S.size() + 2, false);   // mex <= number of moves
        for (int s : S)
            if (s <= n && g[n - s] < (int)seen.size())
                seen[g[n - s]] = true;              // mark grundy of each
reachable state
        int m = 0;
```

```
        while (seen[m]) m++;                          // mex
        g[n] = m;
    }
    return g;
}

// Splitting example: a move removes 1 or 2 ADJACENT pins from a row
(Kayles).
// Removing leaves two rows of sizes a and b, so the next state's grundy =
g[a] ^ g[b].
vector<int> kayles(int maxN) {
    vector<int> g(maxN + 1, 0);
    for (int n = 1; n <= maxN; n++) {
        set<int> seen;
        for (int take = 1; take <= 2 && take <= n; take++)
            for (int a = 0; a + take <= n; a++)      // a pins left of the
gap
                seen.insert(g[a] ^ g[n - take - a]);
        int m = 0;
        while (seen.count(m)) m++;
        g[n] = m;
    }
    return g;
}

// Whole game: XOR of grundy of every pile
bool firstWins(const vector<int>& piles, const vector<int>& g) {
    int x = 0;
    for (int p : piles) x ^= g[p];
    return x != 0;
}
```

Known grundy formulas

Game (one pile of n)	Grundy value g(n)
Nim: take any amount	n
Take 1..k	n % (k+1)
Take any amount from a set S	Compute with DP; always becomes periodic, so find the period
Lasker's Nim: take any amount, OR split into two non-empty piles	n%4 == 1 or 2: n; n%4 == 3: n+1; n%4 == 0: n-1

Game (one pile of n)	Grundy value g(n)
Kayles: remove 1 or 2 adjacent pins from a row	Compute; periodic with period 12 from about n = 71
Grundy's game: split into two UNEQUAL piles	No known formula; compute with DP
Take a divisor d < n of n, go to n - d (or similar number games)	Compute small values, look for a pattern (often depends on the power of 2 in n)

When n is huge (up to 1e18), print g(n) for n = 0..100 with brute force, find the pattern or the period, then hard-code it.

4. Games on graphs

Any impartial game IS a graph: states are nodes, moves are directed edges. How you solve it depends on whether the graph can cycle.

DAG (no cycles): compute grundy numbers with memoized DFS. With several tokens on the DAG, where each turn you move one token, XOR the grundy values of all token positions.

```
// Grundy number of every node of a DAG (adj = moves). Recursive; raise stack
for huge n.
vector<vector<int>> adj;
vector<int> gr; // -1 = not computed
int grundyDAG(int u) {
    if (gr[u] != -1) return gr[u];
    set<int> s;
    for (int v : adj[u]) s.insert(grundyDAG(v));
    int m = 0;
    while (s.count(m)) m++;
    return gr[u] = m;
}
// tokens at t1..tk: first player wins iff grundyDAG(t1) ^ ... ^
grundyDAG(tk) != 0
```

Graph WITH cycles (answer can be Draw): use retrograde analysis. Start from the end positions and work backwards with a BFS.

1. Every node with no moves is LOSE.
2. A node with an edge to a LOSE node is WIN.
3. A node whose moves ALL lead to WIN nodes is LOSE (track a counter of unresolved moves).

4. Nodes never labelled are DRAW: optimal play loops forever.

```
// res[u]: 0 = draw, 1 = win, -1 = lose, for the player to move at u
vector<int> retrograde(int n, const vector<vector<int>>& adj) {
    vector<vector<int>> radj(n);              // reversed edges
    vector<int> deg(n), res(n, 0);
    for (int u = 0; u < n; u++) {
        deg[u] = adj[u].size();
        for (int v : adj[u]) radj[v].push_back(u);
    }
    queue<int> q;
    for (int u = 0; u < n; u++) if (deg[u] == 0) { res[u] = -1; q.push(u); }
    while (!q.empty()) {
        int v = q.front(); q.pop();
        for (int u : radj[v]) {               // u can move to v
            if (res[u] != 0) continue;
            if (res[v] == -1) { res[u] = 1; q.push(u); }   // move to a
losing node
            else if (--deg[u] == 0) { res[u] = -1; q.push(u); } // every move
wins for opponent
        }
    }
    return res;
}
```

For a two-player chase (cop and robber, cat and mouse) the state is (position1, position2, whose turn). Build that state graph, then run the same BFS.

Trees: Green Hackenbush (colon principle): a rooted tree where a move deletes an edge and everything no longer connected to the root. The grundy value of a node is the XOR over its children of (child's grundy + 1). A bare path of length k has grundy k.

```
int hack(int u, int p, const vector<vector<int>>& tree) {
    int g = 0;
    for (int v : tree[u]) if (v != p) g ^= hack(v, u, tree) + 1;
    return g;
}
```

5. Score games (minimax DP)

When players collect points instead of just winning or losing, store the BEST SCORE DIFFERENCE (my points minus the opponent's points) that the player to move can force. The opponent also plays optimally, so you subtract their best from what you take now.

Classic: take from either end of an array. dp[i][j] = best difference for the player to move on a[i..j].

```
dp[i][j] = \max\big(a_i - dp[i+1][j],\; a_j - dp[i][j-1]\big), \qquad dp[i][i] = a_i
```

```
// Returns the first player's total score when both play optimally (O(n^2))
ll takeFromEnds(const vector<ll>& a) {
    int n = a.size();
    vector<vector<ll>> dp(n, vector<ll>(n, 0));
    for (int i = n - 1; i >= 0; i--) {
        dp[i][i] = a[i];
        for (int j = i + 1; j < n; j++)
            dp[i][j] = max(a[i] - dp[i + 1][j],    // take left end
                           a[j] - dp[i][j - 1]);   // take right end
    }
    ll sum = accumulate(a.begin(), a.end(), 0LL);
    return (sum + dp[0][n - 1]) / 2;   // first = (sum + diff) / 2
}
```

Shortcut: with an EVEN number of elements, the first player can always take all even-indexed or all odd-indexed elements, so the first player never loses.

Small state space: bitmask minimax. When n ⩽ 20 items are taken in any order, the state is the mask of items left. Whose turn it is follows from the number of items taken, so the same "difference" trick works.

```
// dp[mask] = best difference for the player to move when the items in mask are left
ll dpm[1 << 20]; bool done[1 << 20];
ll solve(int mask, const vector<ll>& val) {
    if (mask == 0) return 0;
    if (done[mask]) return dpm[mask];
    ll best = LLONG_MIN;
    for (int i = 0; i < (int)val.size(); i++)
        if (mask >> i & 1)
            best = max(best, val[i] - solve(mask ^ (1 << i), val));
    done[mask] = true;
    return dpm[mask] = best;
}
```

Other patterns that use the same idea

- Alice maximizes and Bob minimizes ONE shared value: write two functions, or one function with a turn flag, and take max on Alice's turn and min on Bob's turn.

- Very large numbers in the state (for example n up to 1e18 with moves n → n/2): memoize with a map. Only O(log n) or O(sqrt n) states are actually reachable.
- Trees: dp over the tree where a node's value depends on whose turn it is at that depth (even depth = player 1, odd depth = player 2).

6. Contest checklist

When you see "both players play optimally, who wins?", go through these steps in order.

1. **Read the ending rule.** Does the player who cannot move lose (normal), or the player who makes the last move lose (misère)? Can the game end in a draw?
2. **Is it impartial?** Same moves for both players means win/lose DP or Sprague-Grundy applies.
3. **Split into independent parts.** Separate piles, rows, tokens or connected components are a sum of games, so XOR their grundy values.
4. **Is it a known game?** Compare with the table in section 2 (Nim, Bash, Wythoff, staircase and the others).
5. **Brute force small cases.** Write the win/lose or grundy DP for small n (0..50) and print the results. Look for a period, a modulus, a power of two or a Fibonacci pattern. This is the most reliable method during a contest.
6. **Prove or at least sanity-check the pattern.** Check that every losing position moves only to winning positions, and every winning position has a move to a losing one.
7. **Check limits.** n up to 1e18 means a formula or pattern. n up to 1e5 piles of size up to 1e5 means grundy DP plus XOR. A state space under about 1e7 means direct DP.

Common mistakes

- Using XOR for MISÈRE games. Sprague-Grundy works only for the normal rule; misère Nim needs its special case.
- Using XOR when the parts are NOT independent (for example, one move affects two piles at once).
- Forgetting that grundy values can be larger than 1. Never store them as bool.
- Mixing up "position is losing" with "first player loses". Check who moves first in the input.
- Recursion depth: grundy DFS on a long chain can overflow the stack, so compute bottom-up or iteratively.
- Precision in Wythoff for large values: use long double, or the integer Beatty-sequence check.
- Mex arrays sized too small. The mex is at most the number of moves, so size the seen array by that.